14
시퀀스 레이블링

13장에서는 문장 전체에 정답 하나를 붙였습니다. 이번에는 문장의 단어마다 정답을 하나씩 붙입니다. 문장에서 사람, 조직, 장소의 이름을 찾아내는 개체명 인식기를 양방향 LSTM으로 만듭니다.

오늘 다루는 것

1. 시퀀스 레이블링: 다 대 다 구조와 양방향 RNN
2. 양방향 LSTM을 이용한 개체명 인식

끝나고 나면 할 수 있는 것

1. 다 대 일과 다 대 다의 차이를 텐서의 모양으로 설명할 수 있다.
2. BIO 표기로 붙은 태그에서 개체명을 읽어 낼 수 있다.
3. 정확도 95%가 왜 생각만큼 좋은 숫자가 아닐 수 있는지 설명할 수 있다.

2026년 10월 13일 화요일 14:30 (10/13 자료 2/3)
실습은 모두 코랩 기본 환경에서 실행됩니다. 실습 3을 실행한 뒤 실습 4를 실행합니다(실습 4는 CPU로 약 10분).

14

시퀀스 레이블링 (Sequence Labeling)

"피터는 서울의 구글에서 일한다"에서 피터는 사람, 서울은 장소, 구글은 조직입니다. 이렇게 단어마다 꼬리표(태그)를 붙이는 일이 시퀀스 레이블링입니다. 품사 태깅(단어마다 명사, 동사, …)과 개체명 인식(단어마다 사람, 장소, …)이 대표적입니다. 13장의 모델에서 두 군데만 바꾸면 됩니다.

왜 배우나: 오늘의 학습 경로



예시는 장 전체에서 개체명 인식입니다. 14-01에서는 "peter works at google in seoul" 한 문장으로 구조와 모양을 보고, 14-02에서는 뉴스 문장 14,041개로 실제 모델을 학습시킵니다.

14-01 시퀀스 레이블링

핵심 한 줄

시퀀스 레이블링은 입력의 단어마다 레이블을 하나씩 붙이는 다 대 다 문제이고, 앞뒤 문맥을 모두 보기 위해 양방향 RNN의 모든 시점의 출력을 출력층에 넣는다.

왜 배우나

13장의 텍스트 분류와 무엇이 같고 무엇이 다른지를 먼저 정리합니다. 데이터의 모양, RNN의 어느 출력을 쓰는지, 손실을 어떻게 계산하는지 세 가지가 달라집니다. 이 세 가지를 작은 예로 확인해 두면 14-02의 긴 코드가 쉽게 읽힙니다.

1) 시퀀스 레이블링과 훈련 데이터

정의

- 시퀀스 레이블링(sequence labeling): 입력 시퀀스 $X = [x_1, x_2, \dots, x_n]$의 원소마다 레이블 $y = [y_1, y_2, \dots, y_n]$을 하나씩 붙이는 일. 태깅(tagging)이라고도 합니다.
- 지도 학습입니다. 단어의 줄이 X, 태그의 줄이 y이고, **한 문장 안에서 X와 y의 길이가 같습니다.** x_3의 정답이 y_3입니다(병렬 구조).
- 단어도 태그도 번호로 바꾸고(정수 인코딩), 길이를 맞추려고 **둘 다** 패딩합니다.

	텍스트 분류 (13장)	시퀀스 레이블링 (14장)
정답	문장마다 하나	단어마다 하나
y의 모양	(문장 수,)	(문장 수, 단어 수)
RNN의 구성	다 대 일	다 대 다
쓰는 출력	마지막 시점의 은닉 상태	모든 시점의 은닉 상태
패딩	X만	X와 y 모두

예시 적용

X (단어)	y (태그)	길이
[peter, works, at, google, in, seoul]	[B-PER, O, O, B-ORG, O, B-LOC]	6
[peter, works, in, seoul]	[B-PER, O, O, B-LOC]	4

google과 B-ORG가 한 쌍입니다. 둘째 문장은 길이가 4라서 X와 y 모두 뒤에 패딩 2칸을 붙여 (2, 6) 모양의 텐서 두 개로 만듭니다. 태그의 뜻(B-, O)은 14-02에서 봅니다.

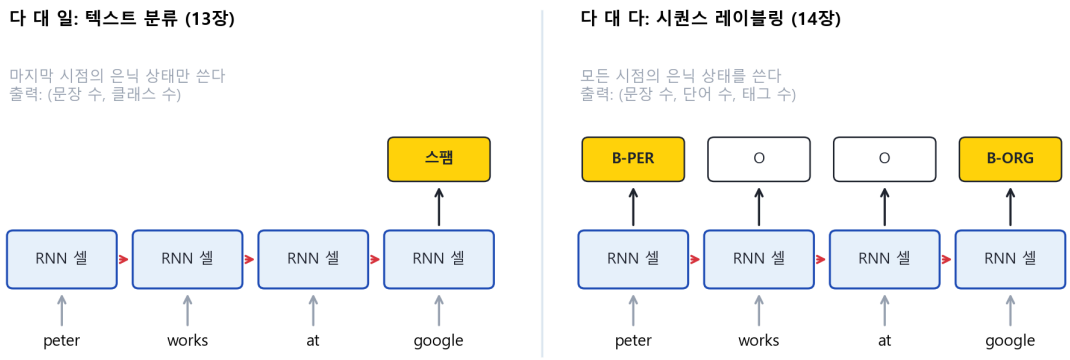


그림 1 왼쪽은 마지막 시점에서만 답을 내고(다 대 일), 오른쪽은 모든 시점에서 답을 낸다(다 대 다)

2) 양방향 RNN과 다 대 다 모델

정의

$$(N,\ L) \xrightarrow{\text{Embedding}} (N,\ L,\ D) \xrightarrow{\text{BiRNN}} (N,\ L,\ 2H) \xrightarrow{\text{Linear}} (N,\ L,\ K)$$

N 문장 수, L 문장의 길이(단어 수), D 임베딩 차원, H 은닉 크기, K 태그의 수입니다. 13장과 달리 L이 끝까지 남습니다. 단어마다 K개의 점수가 나옵니다.

- 양방향 RNN(bidirectional RNN): 문장을 앞에서 뒤로 읽는 RNN과 뒤에서 앞으로 읽는 RNN 두 개를 쓰고, 시점마다 두 은닉 상태를 **이어 붙이는** 구조(07-01). 그래서 출력의 크기가 $2H$입니다.

- 파이토치에서는 `nn.RNN`, `nn.LSTM`, `nn.GRU`에 `bidirectional=True`만 넣으면 됩니다. 뒤따르는 선형층의 입력 크기를 $2H$로 바꿔야 합니다.

왜 양방향일까? 단어의 태그는 뒤에 오는 단어에 따라서도 달라집니다. "Washington said …"의 Washington은 사람이고 "Washington is a city"의 Washington은 장소입니다. 앞에서 뒤로만 읽으면 Washington 자리에서는 아직 뒤의 단어를 보지 못했습니다. 분류에서는 마지막 시점에 문장 전체를 다 읽은 상태였지만, 태깅은 모든 자리에서 답을 내야 하므로 자리마다 양쪽 문맥이 필요합니다.

손실은 어떻게 계산할까? 단어 하나하나가 "태그 K개 중 하나를 고르는" 분류 문제입니다. 그래서 점수 (N, L, K)를 $(N \times L,\ K)$로, 정답 (N, L)을 $(N \times L,)$로 펴서 크로스 엔트로피를 계산합니다. 이때 패딩 자리는 정답이 없으므로 `CrossEntropyLoss(ignore_index=0)`으로 손실에서 뺍니다. 빼지 않으면 모델이 "패딩 자리에는 패딩 태그"를 맞히는 데 힘을 쓰고, 손실도 실제보다 좋아 보입니다.

예시 적용

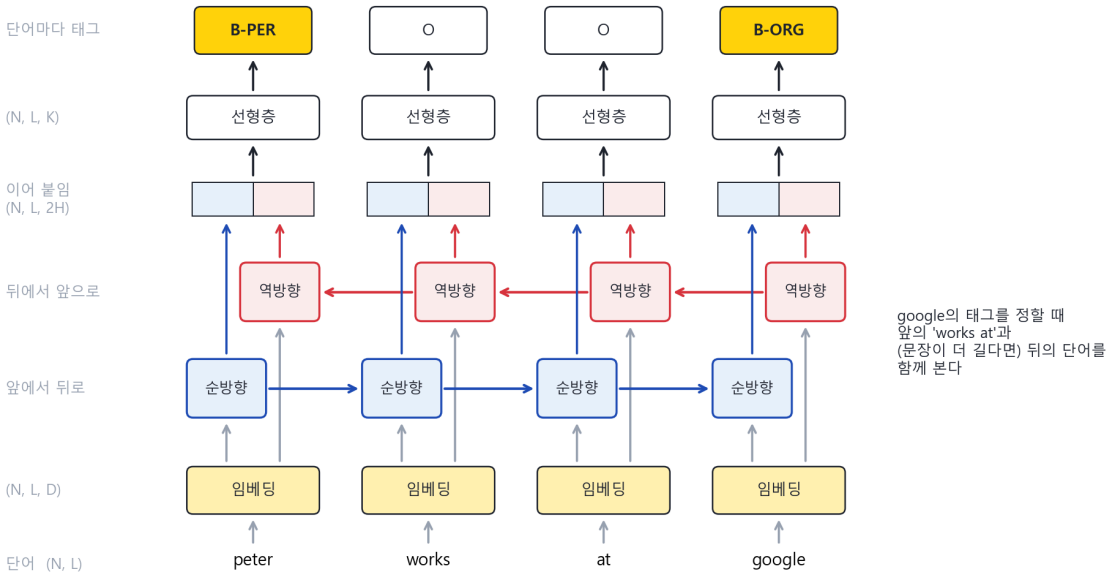


그림 2 양방향 LSTM 태거. 단어마다 순방향(파랑)과 역방향(빨강)의 은닉 상태를 이어 붙여 선형층에 넣고, 단어마다 태그를 하나씩 낸다

위의 두 문장을 $D = 8$, $H = 16$, $K = 5$인 모델에 넣습니다(실습 1).

단계	모양	뜻
입력 X, 정답 y	(2, 6), (2, 6)	문장 2개, 단어 6개씩
임베딩	(2, 6, 8)	
양방향 LSTM의 모든 시점	(2, 6, 32)	16 × 2. 앞 16칸이 순방향, 뒤 16칸이 역방향
출력층	(2, 6, 5)	단어마다 태그 5개의 점수
손실 계산을 위해 펼침	(12, 5), (12,)	단어 12개(패딩 2개 포함)

- 순방향의 마지막 칸과 역방향의 첫 칸이 각각 h_n의 두 행과 같습니다(True, True). 순방향은 마지막 단어에서, 역방향은 첫 단어에서 문장을 다 읽은 상태가 됩니다.
- 패딩을 뺀 손실 1.6773은 실제 단어 10개의 손실만 평균한 값과 같습니다. 패딩까지 넣으면 1.6617로 달라집니다.

비유

영화 한 편의 총평과 장면별 메모

평론가가 영화를 다 보고 별점 하나를 매기는 것이 분류라면, 편집자가 장면마다 "액션", "회상", "대화"라고 메모를 붙이는 것이 시퀀스 레이블링입니다. 어떤 장면이 회상인지는 앞 장면만 봐서는 알기 어렵고, 뒤에 "현재로 돌아오는 장면"이 나와야 분명해집니다. 그래서 편집자는 영화를 앞으로도 뒤로도 돌려 봅니다.

영화 전체의 별점 = = 다 대 일 (분류)
장면마다 붙이는 메모 = = 다 대 다 (태깅)
앞으로 돌려 보기 = = 순방향 RNN
뒤로 돌려 보기 = = 역방향 RNN

실습 1 양방향 LSTM 태거의 모양과 패딩을 뺀 손실

```python
import torch
import torch.nn as nn

torch.manual_seed(0)

# 문장 2개와 단어마다의 태그 (둘째 문장은 짧아서 패딩 2칸)
words = {"<PAD>": 0, "peter": 1, "works": 2, "at": 3, "google": 4, "in": 5,
         "seoul": 6}
tags = {"<PAD>": 0, "O": 1, "B-PER": 2, "B-ORG": 3, "B-LOC": 4}
X = torch.tensor([[1, 2, 3, 4, 5, 6],      # peter works at google in seoul
                  [1, 2, 5, 6, 0, 0]])     # peter works in seoul <PAD> <PAD>
y = torch.tensor([[2, 1, 1, 3, 1, 4],      # B-PER O O B-ORG O B-LOC
                  [2, 1, 1, 4, 0, 0]])     # B-PER O O B-LOC <PAD> <PAD>
print("X:", tuple(X.shape), " y:", tuple(y.shape), "← 입력과 정답의 모양이 같다")

embedding = nn.Embedding(len(words), 8, padding_idx=0)
lstm = nn.LSTM(8, 16, batch_first=True, bidirectional=True)   # 양방향
fc = nn.Linear(16 * 2, len(tags))                             # 은닉 크기 × 2

e = embedding(X)
out, (h_n, c_n) = lstm(e)
logits = fc(out)
print("임베딩          :", tuple(e.shape), "= (문장, 단어, 임베딩 차원)")
print("LSTM 모든 시점 :", tuple(out.shape), "= (문장, 단어, 은닉 크기 × 2)")
print("출력층 점수     :", tuple(logits.shape), "= (문장, 단어, 태그 수)")
print("예측 태그 번호 :", tuple(logits.argmax(dim=-1).shape), "← 단어마다 하나씩")

# 양방향 출력의 앞 절반은 순방향, 뒤 절반은 역방향의 은닉 상태다
forward_last = out[0, -1, :16]     # 순방향: 마지막 단어에서 문장 전체를 읽은 상태
backward_first = out[0, 0, 16:]    # 역방향: 첫 단어에서 문장 전체를 읽은 상태
print("\n순방향의 마지막 = h_n[0]?", torch.allclose(forward_last, h_n[0, 0]))
print("역방향의 첫 칸  = h_n[1]?", torch.allclose(backward_first, h_n[1, 0]))

# 손실: (문장, 단어, 태그)를 (문장 × 단어, 태그)로 펴서 단어마다 계산한다
flat_logits, flat_y = logits.view(-1, len(tags)), y.view(-1)
print("\n펼친 모양:", tuple(flat_logits.shape), tuple(flat_y.shape))
loss_all = nn.CrossEntropyLoss()(flat_logits, flat_y)
loss_ignore = nn.CrossEntropyLoss(ignore_index=0)(flat_logits, flat_y)
print(f"패딩까지 포함한 손실 {loss_all:.4f} / 패딩을 뺀 손실 {loss_ignore:.4f}")

# 패딩을 뺀 손실 = 실제 단어 10개의 손실만 평균한 것
per_word = nn.CrossEntropyLoss(reduction="none")(flat_logits, flat_y)
real = flat_y != 0
print("실제 단어 수:", real.sum().item(),
      " 직접 평균:", f"{per_word[real].mean():.4f}")
```

실행 결과

```
X: (2, 6)  y: (2, 6) ← 입력과 정답의 모양이 같다
임베딩          : (2, 6, 8) = (문장, 단어, 임베딩 차원)
LSTM 모든 시점 : (2, 6, 32) = (문장, 단어, 은닉 크기 × 2)
출력층 점수     : (2, 6, 5) = (문장, 단어, 태그 수)
예측 태그 번호 : (2, 6) ← 단어마다 하나씩

순방향의 마지막 = h_n[0]? True
역방향의 첫 칸  = h_n[1]? True

펼친 모양: (12, 5) (12,)
패딩까지 포함한 손실 1.6617 / 패딩을 뺀 손실 1.6773
실제 단어 수: 10  직접 평균: 1.6773
```

직접 해 보기 `bidirectional=True`를 지우면 어느 줄에서 오류가 나나요? 오류 없이 돌아가게 하려면 `fc`의 입력 크기를 얼마로 바꿔야 하나요?

확인 문제

14-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 시퀀스 레이블링에서 한 문장의 단어 수와 태그 수는 같다. (O / X) 이름 _____

2 시퀀스 레이블링에서는 RNN의 마지막 시점의 은닉 상태만 출력층에 넣는다. (O / X) 이름 _____

3 은닉 크기가 16인 양방향 LSTM의 시점마다의 출력 크기는? 이름 _____

① 16 ② 32 ③ 8

4 양방향 RNN을 쓰는 이유는? 이름 _____

① 단어의 앞뒤 문맥을 모두 보려고 ② 계산을 빠르게 하려고 ③ 패딩을 없애려고

5 패딩 자리를 손실에서 빼려면 `CrossEntropyLoss`에 _____`=0`을 넣는다. 이름 _____

6 모든 시점에서 출력을 내는 RNN의 구성을 다 대 _____라고 한다. 이름 _____

7 텍스트 분류와 시퀀스 레이블링의 차이를 영화의 별점과 장면별 메모에 빗대어 설명해 보세요. 이름 _____

원문: 14-01 시퀀스 레이블링 — https://wikidocs.net/66108 (교재에는 실습이 없어 모양을 확인하는 실습을 추가했습니다)

14-02 양방향 LSTM을 이용한 개체명 인식

핵심 한 줄

개체명 인식은 문장에서 사람·조직·장소 같은 이름을 찾아 유형을 붙이는 일이고, BIO 표기로 단어마다 태그를 붙인 데이터를 양방향 LSTM으로 학습한다.

왜 배우나

뉴스에서 회사 이름만 뽑기, 문서에서 날짜와 금액 찾기, 챗봇이 "내일 서울 날씨"에서 날짜와 장소를 알아듣기가 모두 개체명 인식입니다. 14-01의 구조를 실제 데이터에 적용하고, 결과를 평가할 때 조심할 점(정확도의 함정)을 확인합니다.

1) 개체명 인식과 BIO 표기

정의

- 개체명(named entity): 사람, 조직, 장소, 시간처럼 이름을 가진 대상.
- 개체명 인식(NER, Named Entity Recognition): 글에서 개체명을 찾고 그 유형을 맞히는 일.
- BIO 표기: 개체명이 여러 단어일 수 있으므로, 단어마다 세 가지 가운데 하나를 붙입니다. **B**(Begin) 개체명이 시작하는 단어, **I**(Inside) 개체명이 이어지는 단어, **O**(Outside) 개체명이 아닌 단어. B와 I 뒤에는 유형을 붙입니다(B-PER, I-ORG).

왜 B와 I를 나눌까? 유형만 붙이면 "european commission"이 조직 하나인지 조직 둘이 붙어 있는 것인지 구분할 수 없습니다. B가 나오면 새 개체명이 시작된 것이고, I가 이어지는 동안은 같은 개체명입니다.

유형	뜻	예
PER	사람 (person)	peter blackburn
ORG	조직 (organization)	european commission
LOC	장소 (location)	brussels
MISC	그 밖의 이름 (국적, 행사 등)	german, british

유형 4개 × (B, I) = 8개에 O를 더해 태그는 9개이고, 패딩용 번호 0까지 넣으면 출력층의 크기는 10입니다.

예시 적용

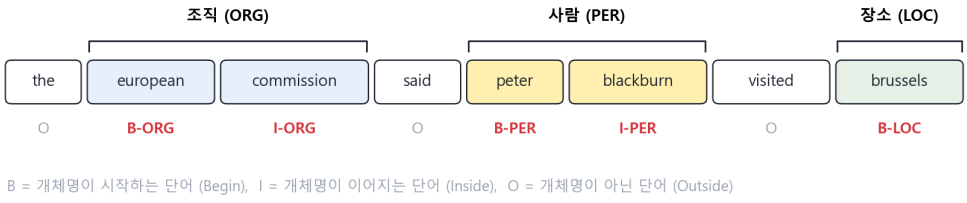


그림 3 BIO 표기. B로 시작해 I가 이어지는 동안이 하나의 개체명이다

이미 만들어진 도구로 먼저 해 보기(실습 2). NLTK의 `ne_chunk`는 품사 태깅 결과를 받아 개체명을 찾아 줍니다.

- "James is working at Disney in London" → James(PERSON), Disney(ORGANIZATION), London(GPE, 지명)
- 같은 문장을 소문자로 쓰면("james is working at disney in london") **하나도 찾지 못합니다.** 품사 태거가 james를 복수 명사(NNS)로, disney와 london을 보통 명사(NN)로 봤기 때문입니다. 이 도구는 대문자에 크게 의존합니다.
- "Peter Blackburn"은 사람 하나인데 Peter와 Blackburn 두 사람으로 나눴습니다.

우리가 만들 모델은 소문자로 바꾼 데이터로 학습합니다. 대문자라는 단서 없이 단어와 문맥만으로 맞혀야 합니다.

2) 데이터 준비

정의

흐름은 13-02와 같고, 정답도 시퀀스라는 점만 다릅니다.

$$\text{file} \rightarrow (\text{words},\ \text{tags}) \rightarrow \text{train / valid / test} \rightarrow \text{vocab} \rightarrow \text{integer encoding} \rightarrow \text{padding} \rightarrow X\,(N, L),\ y\,(N, L)$$

- 단어 집합(단어 → 번호)과 태그 집합(태그 → 번호)을 따로 만듭니다.
- 번호 0은 양쪽 모두 패딩입니다. 단어의 1번은 <UNK>입니다. 태그에는 <UNK>가 없습니다(태그는 정해진 9개뿐이므로).
- X와 y를 같은 길이로 패딩합니다.

예시 적용

데이터는 영어 뉴스 문장에 사람이 개체명 태그를 붙여 둔 CoNLL-2003입니다(실습 3). 한 줄에 단어 하나와 그 태그가 있고, 빈 줄이 문장의 끝입니다.

항목	값
전체 문장	14,041개
학습 / 검증 / 테스트	8,985 / 2,247 / 2,809
단어 집합의 크기	16,744 (<PAD>, <UNK> 포함)
태그 집합의 크기	10 (<PAD> 포함)
문장 길이	최대 78, 평균 14.5 → 80으로 패딩
텐서의 모양	X_train (8985, 80), y_train (8985, 80)

첫 문장 "eu rejects german call to boycott british lamb ."의 태그는 [B-ORG, O, B-MISC, O, O, O, B-MISC, O, O]이고, BIO를 읽어 개체명을 꺼내면 eu(ORG), german(MISC), british(MISC)입니다.

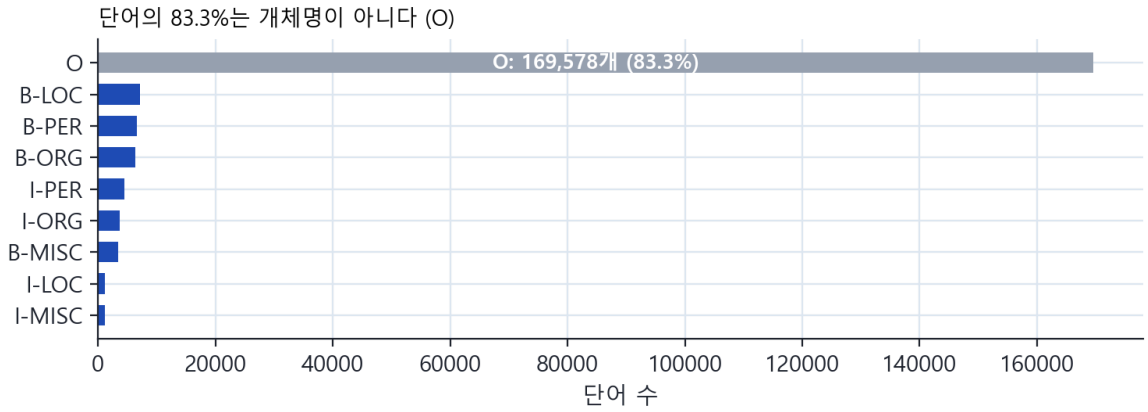


그림 4 태그별 단어 수. O가 169,578개로 전체의 83.3%다

눈여겨볼 것. 단어의 83.3%가 O입니다. 개체명은 문장에 드문드문 나옵니다. 이 불균형이 평가에서 문제가 됩니다.

3) 모델, 학습, 평가

정의

$$(N,\ 80) \xrightarrow{\texttt{Embedding}(V,\,100)} (N,\ 80,\ 100) \xrightarrow{\texttt{LSTM(2 layers, bidirectional)}} (N,\ 80,\ 2H) \xrightarrow{\texttt{Linear}(2H,\,10)} (N,\ 80,\ 10)$$

- 14-01의 구조에서 LSTM을 **2층**으로 쌓았습니다(`num_layers=2` , 07-01의 깊은 RNN).
- 손실: `CrossEntropyLoss(ignore_index=0)` . 정확도를 잴 때도 패딩 자리는 뺍니다(`mask = y ≠ 0`).
- 학습 방법은 13-02와 같습니다. 에포크마다 검증 손실을 재고, 가장 낮을 때의 모델을 저장합니다.

정확도의 함정. 정확도(accuracy)는 "맞힌 단어 수 ÷ 전체 단어 수"입니다. 단어의 83%가 O라면, 아무것도 배우지 않고 **모든 단어에 O라고 답하기만 해도** 정확도가 83%입니다. 이 모델은 개체명을 하나도 찾지 못하는데도 그렇습니다. 06-01에서 본 것처럼, 한쪽이 훨씬 많은 데이터에서는 정확도만 보면 안 됩니다. 그래서 이 자료는 **개체명 단어(정답이 O가 아닌 단어)만의 정확도**를 함께 잽니다.

실제 연구에서는 개체명 단위의 정밀도·재현율·F1(06-01)을 씁니다. 여러 단어로 된 개체명은 시작부터 끝까지 모두 맞아야 맞힌 것으로 칩니다.

예시 적용

양방향 LSTM과, 비교를 위해 단방향 LSTM을 같은 조건으로 3 에포크씩 학습했습니다(실습 4). CPU에서 돌리려고 은닉 크기를 128로 줄였습니다(교재는 256, 양방향만 학습).

테스트 결과	전체 단어 정확도	개체명 단어 정확도
모든 단어를 O로 찍기	0.8331	0
양방향 LSTM	0.9466	0.7280
단방향 LSTM	0.9515	0.7420
교재 (양방향, 은닉 256)	0.9566	재지 않음

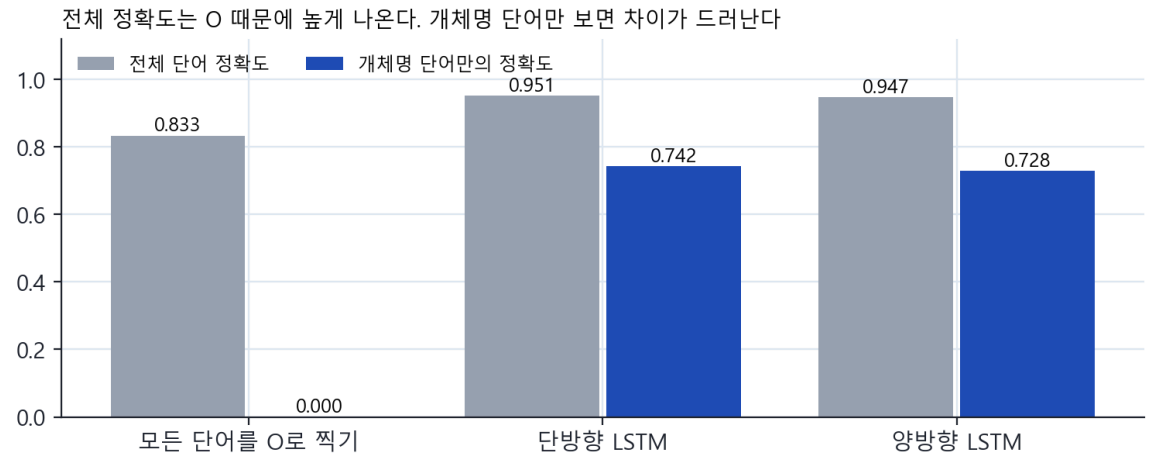


그림 5 전체 정확도는 95% 안팎이지만, 개체명 단어만 보면 73~74%다

- **정확도 0.95의 실제 뜻.** 아무것도 하지 않아도 0.83에서 시작합니다. 개체명 단어만 보면 네 개 중 한 개를 틀립니다. 같은 모델인데 어떤 숫자를 보느냐에 따라 인상이 크게 다릅니다.
- **예상과 다른 결과.** 테스트에서 단방향(0.7420)이 양방향(0.7280)보다 조금 높게 나왔습니다. 양방향이 더 나쁜 모델이라는 뜻은 아닙니다. 아래 그림에서 이유를 봅니다.

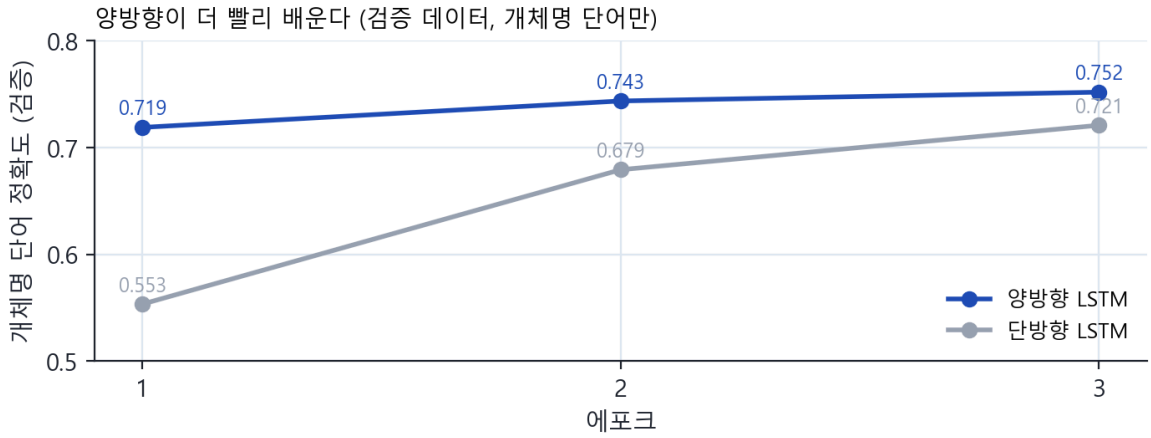


그림 6 에포크마다의 검증 결과(개체명 단어 정확도). 같은 에포크에서는 늘 양방향이 높다

에포크	양방향: 검증 손실	개체명 단어 정확도	단방향: 검증 손실	개체명 단어 정확도
1	0.2058 (저장)	0.7186	0.2963	0.5534
2	0.2136	0.7433	0.2318	0.6791
3	0.2485	0.7516	0.2306 (저장)	0.7206

- 같은 에포크끼리 비교하면 양방향이 항상 높습니다. 첫 에포크에서 0.72 대 0.55로 차이가 가장 큽니다. 양방향이 더 빨리 배웁니다.
- 그런데 양방향은 검증 **손실**이 1 에포크에서 가장 낮고 그 뒤로 올랐습니다. 그래서 저장된 것은 1 에포크의 모델(0.7186)입니다. 단방향은 3 에포크의 모델(0.7206)이 저장됐습니다. 테스트에서 비교한 것은 "1번 학습한 양방향"과 "3번 학습한 단방향"입니다.
- 양방향은 손실이 오르는 동안에도 정확도는 계속 올랐습니다(0.7186 → 0.7516). 손실은 "틀릴 때 얼마나 확신하고 틀렸는가"까지 반영하므로, 맞히는 개수는 늘어도 손실은 나빠질 수 있습니다. 무엇을 기준으로 모델을 고르느냐에 따라 결과가 달라집니다.

예측해 보기. 테스트 문장 "feyenoord rotterdam suffered … against de graafschap doetinchem ."에서 feyenoord rotterdam을 조직(B-ORG, I-ORG)으로 맞혔습니다. graafschap은 B-ORG가 정답인데 I-ORG로 틀렸습니다. 바로 앞의 de를 조직 이름의 시작으로 보지 못했는데 graafschap은 이어지는 단어라고 답한 것입니다.

- "Peter works at Samsung Electronics in Seoul" → Peter(B-PER), Seoul(B-LOC)은 맞혔고, Samsung은 B-MISC로 틀렸으며 Electronics는 놓쳤습니다.
- "Jihoon visited the Blue House yesterday" → 하나도 찾지 못했습니다. jihoon은 학습 데이터에 없는 단어라 <UNK>가 됩니다. 1996년 유럽 뉴스로 학습한 모델이라 한국 이름과 회사는 본 적이 없습니다. 15장의 서브워드와 12장의 사전 훈련된 임베딩이 이런 문제를 줄여 줍니다.

비유

기사에 형광펜 칠하기

신문 기사를 읽으며 사람 이름은 노랑, 회사 이름은 파랑, 장소는 초록으로 칠합니다. 이름이 두 단어면 이어서 칠하고, 이름 둘이 붙어 있으면 사이를 끊어 칠합니다. 기사의 대부분은 칠하지 않는 글자입니다. "아무것도 칠하지 않은 학생"도 글자 수로 채점하면 83점을 받습니다. 그래서 칠해야 할 곳을 얼마나 맞혔는지로 채점해야 합니다.

형광펜의 색	= = 개체명의 유형 (PER, ORG, LOC)
칠하기 시작하는 곳	= = B
이어서 칠하는 곳	= = I
칠하지 않는 글자	= = O
글자 수로 채점	= = 전체 정확도

실습 2 NLTK로 개체명 인식 해 보기

```python
import nltk
from nltk import ne_chunk, pos_tag, word_tokenize

for name in ["punkt_tab", "averaged_perceptron_tagger_eng",
             "maxent_ne_chunker_tab", "words"]:
    nltk.download(name, quiet=True)

def show(sentence):
    tagged = pos_tag(word_tokenize(sentence))        # 토큰화 후 품사 태깅
    tree = ne_chunk(tagged)                          # 품사를 보고 개체명 인식
    found = [(" ".join(w for w, _ in t.leaves()), t.label())
             for t in tree if hasattr(t, "label")]
    print(sentence)
    print("  품사  :", " ".join(f"{w}/{t}" for w, t in tagged))
    print("  개체명:", found)

show("James is working at Disney in London")
show("james is working at disney in london")         # 소문자로 쓰면?
show("Peter Blackburn visited the European Commission in Brussels")
```

```
실행 결과
James is working at Disney in London
  품사  : James/NNP is/VBZ working/VBG at/IN Disney/NNP in/IN London/NNP
  개체명: [('James', 'PERSON'), ('Disney', 'ORGANIZATION'), ('London', 'GPE')]
james is working at disney in london
  품사  : james/NNS is/VBZ working/VBG at/IN disney/NN in/IN london/NN
  개체명: []
Peter Blackburn visited the European Commission in Brussels
  품사  : Peter/NNP Blackburn/NNP visited/VBD the/DT European/JJ Commission/NNP in/IN Brussels/NNP
  개체명: [('Peter', 'PERSON'), ('Blackburn', 'PERSON'), ('European Commission', 'ORGANIZATION'), ('Brussels', 'GPE')]
```

직접 해 보기 직접 쓴 영어 문장을 넣어 보세요. 문장의 맨 앞 단어처럼 원래 대문자로 시작하는 보통 명사를 개체명으로 잘못 잡는 경우가 있나요?

실습 3 개체명 인식 데이터를 정수 텐서로 준비하기

```python
# 개체명 인식 데이터(CoNLL-2003)를 내려받아 정수 텐서로 준비한다
import urllib.request
from collections import Counter

import torch
from sklearn.model_selection import train_test_split

url = ("https://raw.githubusercontent.com/ukairia777/tensorflow-nlp-tutorial/"
       "main/12.%20RNN%20Sequence%20Labeling/dataset/train.txt")
urllib.request.urlretrieve(url, filename="train.txt")

# 1) 파일 읽기: 한 줄에 "단어 품사 구 개체명", 빈 줄이 문장의 끝
sentences, ner_tags, words, tags = [], [], [], []
for line in open("train.txt", encoding="utf8"):
    if line.startswith("-DOCSTART") or line.strip() == "":
        if words:
            sentences.append(words)
            ner_tags.append(tags)
            words, tags = [], []
        continue
    parts = line.split()
    words.append(parts[0].lower())                   # 단어는 소문자로
    tags.append(parts[-1])                           # 맨 끝 열이 개체명 태그
print("전체 문장 수:", len(sentences))
print("첫 문장:", sentences[0])
print("태그   :", ner_tags[0])

# 2) BIO 태그에서 개체명 꺼내기: B로 시작해 I가 이어지는 동안이 한 덩어리
def extract(words, tags):
    found, current = [], None
    for w, t in zip(words, tags):
        if t.startswith("B-") or (t.startswith("I-") and current is None):
            current = [t[2:], [w]]
            found.append(current)
        elif t.startswith("I-"):
            current[1].append(w)
        else:
            current = None
```

```
    return [(" ".join(ws), kind) for kind, ws in found]

for i in [0, 12]:
    print(f"  {' '.join(sentences[i])}\n    → {extract(sentences[i], ner_tags[i])}")

# 3) 태그의 분포: 대부분의 단어는 개체명이 아니다(O)
counts = Counter(t for ts in ner_tags for t in ts)
total = sum(counts.values())
print("태그별 단어 수:", dict(counts.most_common()))
print(f"O 태그의 비율: {counts['O'] / total:.1%}")

# 4) 나누기 (교재와 같은 시드) → 단어 집합 → 정수 인코딩 → 패딩
X_train, X_test, y_train, y_test = train_test_split(
    sentences, ner_tags, test_size=0.2, random_state=777)
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train, y_train, test_size=0.2, random_state=777)
print("학습/검증/테스트:", len(X_train), len(X_valid), len(X_test))

word_counts = Counter(w for s in X_train for w in s)
vocab = sorted(word_counts, key=word_counts.get, reverse=True)
word_to_index = {"<PAD>": 0, "<UNK>": 1, **{w: i + 2 for i, w in enumerate(vocab)}}
tag_names = ["<PAD>", "O", "B-PER", "I-PER", "B-ORG", "I-ORG", "B-LOC", "I-LOC",
             "B-MISC", "I-MISC"]
tag_to_index = {t: i for i, t in enumerate(tag_names)}
print("단어 집합의 크기:", len(word_to_index), " 태그 집합의 크기:", len(tag_names))

lengths = [len(s) for s in X_train]
print(f"문장 길이: 최대 {max(lengths)}, 평균 {sum(lengths) / len(lengths):.1f}")
max_len = 80

def encode(data, table, unk=None):
    out = torch.zeros(len(data), max_len, dtype=torch.long)       # 0 = 패딩
    for i, seq in enumerate(data):
        ids = [table.get(x, unk) for x in seq][:max_len]
        out[i, :len(ids)] = torch.tensor(ids)
    return out

data = {"word_to_index": word_to_index, "tag_names": tag_names}
for name, X, y in [("train", X_train, y_train), ("valid", X_valid, y_valid),
                   ("test", X_test, y_test)]:
    data[f"X_{name}"] = encode(X, word_to_index, unk=1)
    data[f"y_{name}"] = encode(y, tag_to_index)       # 정답(태그)도 패딩한다
data["test_sentences"], data["test_tags"] = X_test, y_test
torch.save(data, "ner_data.pt")
print("X_train:", tuple(data["X_train"].shape),
      " y_train:", tuple(data["y_train"].shape))
print("첫 문장 X:", data["X_train"][0][:12].tolist(), "...")
print("첫 문장 y:", data["y_train"][0][:12].tolist(), "...")
```

```
실행 결과
전체 문장 수: 14041
첫 문장: ['eu', 'rejects', 'german', 'call', 'to', 'boycott', 'british', 'lamb', '.']
태그   : ['B-ORG', 'O', 'B-MISC', 'O', 'O', 'O', 'B-MISC', 'O', 'O']
  eu rejects german call to boycott british lamb .
    → [('eu', 'ORG'), ('german', 'MISC'), ('british', 'MISC')]
  only france and britain backed fischler 's proposal .
    → [('france', 'LOC'), ('britain', 'LOC'), ('fischler', 'PER')]
태그별 단어 수: {'O': 169578, 'B-LOC': 7140, 'B-PER': 6600, 'B-ORG': 6321, 'I-PER': 4528, 'I-ORG': 3704, 'B-MISC': 3438, 'I-
LOC': 1157, 'I-MISC': 1155}
O 태그의 비율: 83.3%
학습/검증/테스트: 8985 2247 2809
단어 집합의 크기: 16744  태그 집합의 크기: 10
문장 길이: 최대 78, 평균 14.5
X_train: (8985, 80)  y_train: (8985, 80)
첫 문장 X: [1260, 3215, 117, 17, 21, 123, 56, 539, 23, 0, 0, 0] ...
첫 문장 y: [4, 5, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0] ...
```

직접 해 보기 `extract` 함수에 `["new", "york", "times"]`와 `["B-LOC", "I-LOC", "B-ORG"]`를 넣으면 무엇이 나오나요? 태그를 `["B-ORG", "I-ORG", "I-ORG"]`로 바꾸면요?

실습 4 양방향 LSTM으로 개체명 인식 모델 학습하기

```
# 앞 실습(데이터 준비)을 먼저 실행해 둔다 (CPU로 약 10분)
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

data = torch.load("ner_data.pt")
word_to_index, tag_names = data["word_to_index"], data["tag_names"]
vocab_size, num_tags = len(word_to_index), len(tag_names)

class NERTagger(nn.Module):
    def __init__(self, bidirectional, embedding_dim=100, hidden_dim=128):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, num_layers=2,
                            batch_first=True, bidirectional=bidirectional)
        # 양방향이면 은닉 상태 두 개가 이어 붙어 크기가 2배가 된다
        self.fc = nn.Linear(hidden_dim * (2 if bidirectional else 1), num_tags)

    def forward(self, x):
        out, _ = self.lstm(self.embedding(x))   # (배치, 80, 은닉 크기 × 방향 수)
        return self.fc(out)                     # (배치, 80, 태그 수): 단어마다 점수

criterion = nn.CrossEntropyLoss(ignore_index=0)       # 패딩 자리는 손실에서 뺀다

def evaluate(model, X, y):
```

```
    """패딩을 뺀 손실, 전체 단어 정확도, 개체명 단어(O가 아닌 것)만의 정확도"""
    model.eval()
    with torch.no_grad():
        batches = DataLoader(TensorDataset(X), batch_size=256)
        logits = torch.cat([model(bx) for (bx,) in batches])
    loss = criterion(logits.view(-1, num_tags), y.view(-1)).item()
    pred = logits.argmax(dim=-1)
    real, entity = y != 0, y > 1                        # 1번 태그가 O
    acc = (pred == y)[real].float().mean().item()
    entity_acc = (pred == y)[entity].float().mean().item()
    return loss, acc, entity_acc

def fit(model, epochs, path):
    optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
    loader = DataLoader(TensorDataset(data["X_train"], data["y_train"]),
                        batch_size=32, shuffle=True)
    best, history = float("inf"), []
    for epoch in range(1, epochs + 1):
        model.train()
        for bx, by in loader:
            loss = criterion(model(bx).view(-1, num_tags), by.view(-1))
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
        result = evaluate(model, data["X_valid"], data["y_valid"])
        history.append(result)
        saved = ""
        if result[0] < best:
            best, saved = result[0], "  ← 저장"
            torch.save(model.state_dict(), path)
        print(f"  에포크 {epoch}: 검증 손실 {result[0]:.4f},",
              f"정확도 {result[1]:.4f},",
              f"개체명 단어 정확도 {result[2]:.4f}{saved}")
    model.load_state_dict(torch.load(path))
    return history

results = {}
for name, bidirectional in [("양방향 LSTM", True), ("단방향 LSTM", False)]:
    print(f"[{name}]")
    torch.manual_seed(0)
    model = NERTagger(bidirectional)
    history = fit(model, epochs=3, path=f"best_{int(bidirectional)}.pth")
    test = evaluate(model, data["X_test"], data["y_test"])
    print(f"  테스트: 손실 {test[0]:.4f}, 정확도 {test[1]:.4f},",
          f"개체명 단어 정확도 {test[2]:.4f}")
    results[name] = {"history": history, "test": test}
    if bidirectional:
        bi_model = model

# 모든 단어를 O라고 답하기만 해도 나오는 정확도
y_test = data["y_test"]
baseline = ((y_test == 1).sum() / (y_test != 0).sum()).item()
print(f"\n모든 단어를 O로 찍을 때의 정확도: {baseline:.4f}")
results["baseline"] = baseline
torch.save(results, "ner_results.pt")

def predict(words):
    ids = [word_to_index.get(w.lower(), 1) for w in words]
    bi_model.eval()
    with torch.no_grad():
        pred = bi_model(torch.tensor([ids])).argmax(dim=-1)[0]
    return [tag_names[i] for i in pred.tolist()]

words, answer = data["test_sentences"][0], data["test_tags"][0]
pred = predict(words)
print("\n테스트 문장:", " ".join(words))
print("단어            정답     예측")
for w, a, p in zip(words, answer, pred):
    if a != "O" or p != "O":
        print(f"{w:15s} {a:8s} {p:8s} {'' if a == p else '← 틀림'}")

for sentence in ["Peter works at Samsung Electronics in Seoul",
                 "Jihoon visited the Blue House yesterday"]:
    words = sentence.split()
    found = [(w, t) for w, t in zip(words, predict(words)) if t != "O"]
    print(sentence, "→", found)
```

실행 결과

```
[양방향 LSTM]
  에포크 1: 검증 손실 0.2058, 정확도 0.9449, 개체명 단어 정확도 0.7186  ← 저장
  에포크 2: 검증 손실 0.2136, 정확도 0.9522, 개체명 단어 정확도 0.7433
  에포크 3: 검증 손실 0.2485, 정확도 0.9530, 개체명 단어 정확도 0.7516
  테스트: 손실 0.1948, 정확도 0.9466, 개체명 단어 정확도 0.7280
[단방향 LSTM]
  에포크 1: 검증 손실 0.2963, 정확도 0.9187, 개체명 단어 정확도 0.5534  ← 저장
  에포크 2: 검증 손실 0.2318, 정확도 0.9412, 개체명 단어 정확도 0.6791  ← 저장
  에포크 3: 검증 손실 0.2306, 정확도 0.9478, 개체명 단어 정확도 0.7206  ← 저장
  테스트: 손실 0.2149, 정확도 0.9515, 개체명 단어 정확도 0.7420

모든 단어를 O로 찍을 때의 정확도: 0.8331

테스트 문장: feyenoord rotterdam suffered an early shock when they went 1-0 down after four minutes against de graafschap
doetinchem .
단어            정답     예측
feyenoord       B-ORG    B-ORG
rotterdam       I-ORG    I-ORG
```

```
graafschap      B-ORG    I-ORG    ← 틀림
doetinchem      I-ORG    I-ORG
Peter works at Samsung Electronics in Seoul → [('Peter', 'B-PER'), ('Samsung', 'B-MISC'), ('Seoul', 'B-LOC')]
Jihoon visited the Blue House yesterday → []
```

직접 해 보기 `fit`에서 저장하는 기준을 검증 손실 대신 개체명 단어 정확도(`result[2]`가 가장 높을 때)로 바꾸면, 양방향과 단방향의 테스트 결과가 어떻게 달라질까요?

확인 문제

14-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 BIO 표기에서 O는 개체명이 아닌 단어에 붙인다. (O / X) 이름 _____

2 정확도가 95%이면 개체명의 95%를 찾아낸 것이다. (O / X) 이름 _____

3 "european commission"(조직)의 태그로 맞는 것은? 이름 _____

① B-ORG, B-ORG ② B-ORG, I-ORG ③ I-ORG, B-ORG

4 실습 데이터에서 O 태그가 차지하는 비율은? 이름 _____

① 약 17% ② 약 50% ③ 약 83%

5 개체명 인식을 영어 약자로 _____이라고 한다. 이름 _____

6 모든 단어를 O로 찍기만 해도 테스트 정확도는 약 _____이다. 이름 _____

7 이 데이터에서 정확도만 보면 왜 안 되는지, 기사에 형광펜 칠하기에 빗대어 설명해 보세요. 이름 _____

원문: 14-02 양방향 LSTM을 이용한 개체명인식 — https://wikidocs.net/217054 (은닉 크기를 256에서 128로, 에포크를 3으로 줄여 CPU에서 실행했습니다. 단방향 LSTM과의 비교, "모두 O로 찍기" 기준선, 개체명 단어만의 정확도는 이 자료에서 추가했습니다. 태그 번호의 순서는 교재와 다릅니다)

오늘의 요약

다시 보기



정의 모음

개념	정의	예시
시퀀스 레이블링	단어마다 레이블을 하나씩 붙이는 일	peter → B-PER
다 대 다	모든 시점의 은닉 상태를 출력층으로	(2, 6, 32) → (2, 6, 5)
양방향 RNN	순방향과 역방향의 은닉 상태를 이어 붙임	16 × 2 = 32
패딩을 뺀 손실	`ignore_index=0`으로 패딩 자리를 뺌	단어 10개만 평균
BIO 표기	B 시작, I 이어짐, O 개체명 아님	B-ORG, I-ORG
정확도의 함정	한쪽이 많으면 찍어도 정확도가 높다	모두 O: 0.8331

다음

다음 자료: 15장 서브워드 토크나이저 (10/13 자료 3/3)

오늘의 모델은 jihoon 같은 처음 보는 단어를 <UNK>로 바꿔 버려서 읽지 못했습니다. 다음 자료에서는 모르는 단어를 아는 조각으로 쪼개어 읽는 방법(BPE)을 배웁니다.

정답과 해설

14-01 시퀀스 레이블링

1. O
2. X — 모든 시점의 은닉 상태를 넣습니다. 마지막만 쓰는 것은 분류(다 대 일)입니다.
3. ② — 순방향 16 + 역방향 16
4. ①
5. ignore_index
6. 다
7. 예시 답: 분류는 영화를 다 보고 별점 하나를 매기는 것처럼 문장 전체에 정답 하나를 내고, 시퀀스 레이블링은 장면마다 메모를 붙이듯 단어마다 정답을 낸다. 장면의 성격은 앞뒤 장면을 다 봐야 알 수 있으므로 양방향으로 읽는다.

14-02 양방향 LSTM을 이용한 개체명 인식

1. O
2. X — 단어의 83%가 O라서, 개체명 단어만 보면 73% 정도였습니다.
3. ②
4. ③ — 83.3%
5. NER
6. 0.83 (0.8331)
7. 예시 답: 기사의 글자 대부분은 형광펜을 칠하지 않는 곳이라, 아무것도 칠하지 않아도 글자 수로 채점하면 83점이 나온다. 칠해야 할 곳을 얼마나 맞혔는지를 따로 봐야 모델이 이름을 실제로 찾는지 알 수 있다.

용어 정리

용어	정의
시퀀스 레이블링 (태깅)	입력의 원소마다 레이블을 하나씩 붙이는 일
다 대 다	모든 시점에서 입력을 받고 모든 시점에서 출력을 내는 구성
양방향 RNN	순방향 RNN과 역방향 RNN의 은닉 상태를 이어 붙여 쓰는 구조
개체명	사람, 조직, 장소처럼 이름을 가진 대상
개체명 인식 (NER)	글에서 개체명을 찾고 유형을 맞히는 일
BIO 표기	개체명의 시작(B), 이어짐(I), 개체명 아님(O)을 단어마다 붙이는 방법
ignore_index	손실을 계산할 때 빼 놓을 정답 번호. 패딩 자리를 빼는 데 쓴다
정확도	맞힌 것의 수 ÷ 전체의 수. 한쪽이 많은 데이터에서는 높게 나온다